Continuous Assessment - Part B ADVANCED DATABASES

B9IS125_2526_TMD1S

Submitted By:

1. **Karnishwar R (20086868)**
2. **Anandha Krishnan Sushandan (20055947)**
3. **Sanu Mathew (20075856)**

ASSESSMENT DETAILS:

GROUP MEMBERS	**1. Karnishwar R (20086868)** **2. Anandha Krishnan Sushandan (20055947)** **3. Sanu Mathew (20075856)**
UNIT TITLE	**ADVANCED DATABASES**
UNIT CODE	**B9IS125**
UNIT LECTURER	**Jagadeeswaran Thangaraj**
LEVEL	**9**
ASSESSMENT TITLE	**CA PART - B**
ASSESSMENT NUMBER	**2**
ASSESSMENT TYPE	**GROUP**
ASSESSMENT WEIGHTING	**70%**
ISSUE DATE	**27 Nov, 2025**
HAND IN DATE	**19 Dec, 2025**
MODE OF SUBMISSION	**ON-LINE VIA MOODLE**

Table of Contents:

SL No	Topic	Page Number
1	Introduction	4
2	Design And Implementation of Event Management Database	7
3	Hybrid Database Features and XML Implementation	23
4	Challenges And Solutions	27
5	Innovation	29
6	Conclusion	29
7	References	30
8	Table Creation Images	31
9	ER Diagram	36

1. INTRODUCTION

1.1 Purpose and Objectives of the Assignment:

- This assignment focuses on implementing the database design that was created during the earlier analysis phase for the ASK Event Booking and Management Company. The main aim of this work was to convert the proposed design into a working database using Microsoft SQL Server.

- We stored core operational data such as client details, events, employees, vendors, and payments using relational tables. Along with this, we used XML to store event-specific information like schedules and special requirements, which vary from one event to another.

- We created the required tables with suitable constraints, inserted sample data, and wrote SQL queries, views, triggers, and stored procedures to apply the business rules. XML data types were also used to store and retrieve hierarchical event data.

- Overall, this project shows how relational and semi-structured data can be used together to build a practical database system that supports day-to-day event management.

1.2 Overview of the Business Scenario:

- ASK Events is an events booking and management company. They arrange weddings, corporate, conferences and private party. To organize such events, they need to organize numerous activities, including client registration, date setting, venue selection, staffing, vendor and payment tracking.

- The company will need to miss out on double-booking of the venues, assigning the correct manager and vendors to a particular event, and maintain the proper records on payments every day. When things are busy, then it is more likely that errors can be made when this is done without a clear system.

- A database system should have the right to hold systematized data, including information about clients, events, employees, vendor services, and payroll. It should also be able to manage semi-structured data such as event schedules and special client requests, which vary with each event.

- In response to these requirements ASK Events employs hybrid database strategy which combines relational tables with XML-based data storage.

1.3 Purpose and Benefits of a Hybrid Database System:

- The hybrid database system utilizes normal tables as well as stores data in XML files which may be modified with ease.
- This would suit well in ASK Event Booking and Management System since event management contains some fixed data as well as data that varies as per the event.
- The normal tables contain well-organized data such as client details, event information and payment details in a clear-cut form.
- XML section is variable data containing event schedules and additional requirements.
- XML implies that we need not continue modifying the database design whenever the details of an event are modified.
- This method keeps the information in the data correct with regulations and automatic procedures, and this enabling the system to expand with additional events being included or evolved into new and complicated ones.
- Requirements and direct queries enable the managers and staff members to easily retrieve the required information.

1.4 Business Rules and Implementation:

A. A venue can only be booked for one event at a specific date and time:

Only one event can take place at a specific date and time in a venue. After being attached to an event in the EVENT table, a venue cannot be booked at the same date and time to be used in another event. This regulation prevents the issue of double-booking and scheduling.

B. Each event must be linked to an existing client:

Each incident has to be linked to an existing client. The event records are required to have a valid Client_ID as a reference to the CLIENT table. This guarantees that events can be created and managed only by registered clients.

C. Only one manager can be assigned to an event:

Every event is allowed to have a single manager. In the EVENT table, an event should have only one Manager_ID, and therefore it is obvious who is in charge.

D. Vendors can be assigned to multiple events, but only through a junction table:

Vendors may collaborate on a wide range of events although they are related to events via the EVENT_VENDOR table. This table allows a vendor to be employed in multiple events, and it tracks his services. This supports many-many relationship

E. Each payment must be linked to a valid event and follow a fixed payment status:

All payments should relate to an actual occasion and be specified in status. All the records of payments should have an existing Event_ID, and the status could only be Pending, Completed, and Cancelled. This ensures that there is the accuracy of payment tracking and prevention of invalid payments.

F. An event must have a specific date and venue before confirmation:

The confirmation of an event cannot be done until the date and venue are established. This is to make sure that the event is well-scheduled before it becomes final.

TABLE CREATION AND ER DIAGRAM SCREENSHOTS ARE ATTACHED AT **SECTION 8 (Table Creation images), SECTION 9 (ER Diagram).**

2. DESIGN AND IMPLEMENTATION OF EVENT MANAGEMENT DATABASE

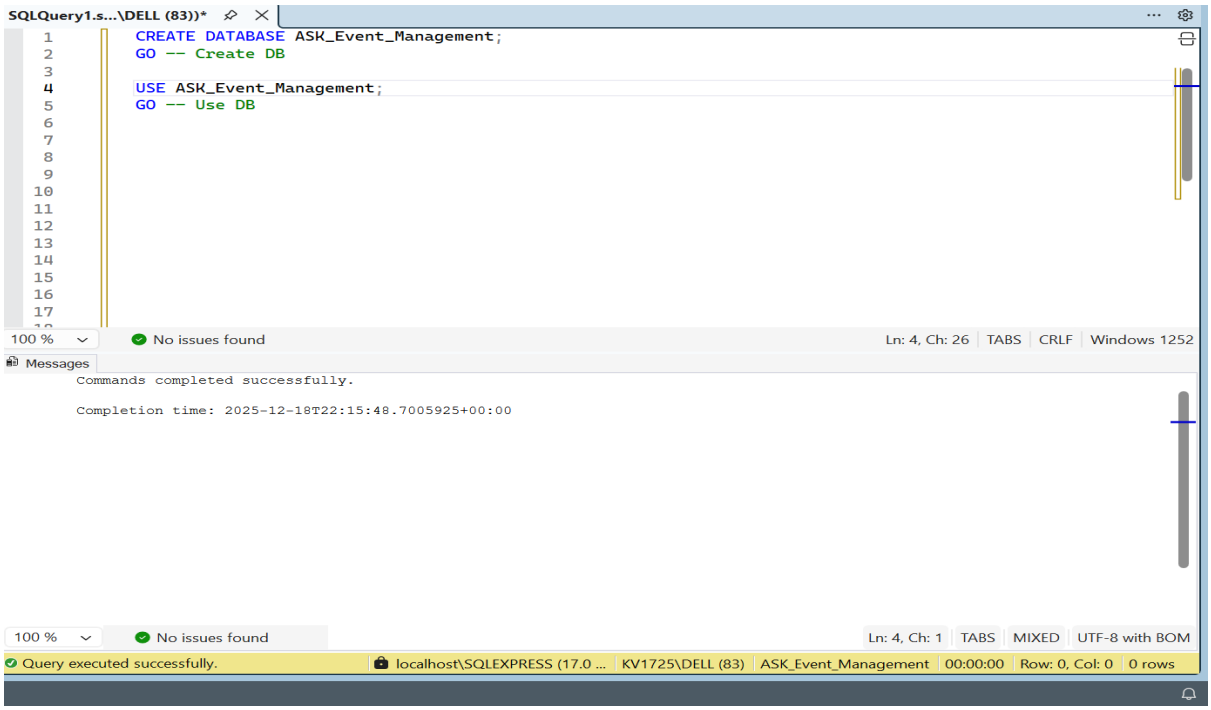


Image 1 – Database Creation and using Database

2.1 Overview of Tables:

● CLIENT:

Carries information regarding the clients of the event system. Primary keys: client_id, client_name, email, phone, address.

```sql
INSERT INTO CLIENT (ClientName, Email, Phone, Address) VALUES
('John O Sullivan','john.osullivan@mail.com','0873214567','Dublin'),
('Emma Fitzgerald','emma.fitzgerald@mail.com','0876543210','Cork'),
('Michael Byrne','michael.byrne@mail.com','0879988776','Galway'),
('Aisha Khan','aisha.khan@mail.com','0877766554','Limerick'),
('David Murphy','david.murphy@mail.com','0874455667','Waterford'),
('Laura Nolan','laura.nolan@mail.com','0878899001','Dublin'),
('Ryan O Connor','ryan.oconnor@mail.com','0872233445','Cork'),
('Sophie Walsh','sophie.walsh@mail.com','0876677889','Galway'),
('Daniel McCarthy','daniel.mccarthy@mail.com','0873344556','Tralee'),
('Priya Patel','priya.patel@mail.com','0875566778','Dublin');
GO

SELECT * FROM CLIENT
```

	Client_ID	ClientName	Email	Phone	Address
1	1	John O Sullivan	john.osullivan@mail.com	0873214567	Dublin
2	2	Emma Fitzgerald	emma.fitzgerald@mail.com	0876543210	Cork
3	3	Michael Byrne	michael.byrne@mail.com	0879988776	Galway
4	4	Aisha Khan	aisha.khan@mail.com	0877766554	Limerick
5	5	David Murphy	david.murphy@mail.com	0874455667	Waterford
6	6	Laura Nolan	laura.nolan@mail.com	0878899001	Dublin
7	7	Ryan O Connor	ryan.oconnor@mail.com	0872233445	Cork
8	8	Sophie Walsh	sophie.walsh@mail.com	0876677889	Galway
9	9	Daniel McCarthy	daniel.mccarthy@mail.com	0873344556	Tralee
10	10	Priya Patel	priya.patel@mail.com	0875566778	Dublin

Image 2 – Client Table

● EMPLOYEE:

The data regarding employees and event managers.

Primary Keys: manager_id, manager_name, position, email, phone number.

```sql
INSERT INTO EMPLOYEE (ManagerName, Email, Phone, Position) VALUES
('Patrick Ryan','patrick.ryan@mail.com','0891112233','Event Manager'),
('Linda Keane','linda.keane@mail.com','0892223344','Senior Coordinator'),
('Brian Doyle','brian.doyle@mail.com','0893334455','Supervisor'),
('Niamh Farrell','niamh.farrell@mail.com','0894445566','Assistant Manager'),
('Sean Murphy','sean.murphy@mail.com','0895556677','Event Manager'),
('Claire O Reilly','claire.oreilly@mail.com','0896667788','Coordinator'),
('Mark Higgins','mark.higgins@mail.com','0897778899','Supervisor'),
('Aoife Brennan','aoife.brennan@mail.com','0898889900','Assistant'),
('Thomas Kelly','thomas.kelly@mail.com','0899990011','Event Manager'),
('Rachel Flynn','rachel.flynn@mail.com','0891011122','Coordinator');
GO

SELECT * FROM EMPLOYEE
```

	Manager_ID	ManagerName	Email	Phone	Position
1	1	Patrick Ryan	patrick.ryan@mail.com	0891112233	Event Manager
2	2	Linda Keane	linda.keane@mail.com	0892223344	Senior Coordinator
3	3	Brian Doyle	brian.doyle@mail.com	0893334455	Supervisor
4	4	Niamh Farrell	niamh.farrell@mail.com	0894445566	Assistant Manager
5	5	Sean Murphy	sean.murphy@mail.com	0895556677	Event Manager
6	6	Claire O Reilly	claire.oreilly@mail.com	0896667788	Coordinator
7	7	Mark Higgins	mark.higgins@mail.com	0897778899	Supervisor
8	8	Aoife Brennan	aoife.brennan@mail.com	0898889900	Assistant
9	9	Thomas Kelly	thomas.kelly@mail.com	0899990011	Event Manager
10	10	Rachel Flynn	rachel.flynn@mail.com	0891011122	Coordinator

Image 3 – Employee Table

● EVENT:

Stores information on the events that have been booked by customers.

Primary Columns: event_id, event_name, event_type, event_date, venue, client_id, manager_id.

SQLQuery1.s...\DELL (83))*

SQLQuery1.sql - localhost\SQLEXPRESS.ASK_Event_Management (KV1725\DELL (83))*

```sql
INSERT INTO EVENT (EventName, EventType, EventDate, Venue, Client_ID, Manager_ID) VALUES
('O Sullivan Wedding','Wedding','2025-03-01','Grand Hall Dublin',1,1),
('Tech Leaders Conference','Conference','2025-03-05','City Convention Centre',2,2),
('Byrne 40th Birthday Celebration','Private','2025-03-10','Seaview Hotel Galway',3,3),
('Corporate Strategy Meeting','Corporate','2025-03-15','Business Park Limerick',4,4),
('Murphy Wedding Reception','Wedding','2025-03-20','Royal Hotel Waterford',5,5),
('Startup Networking Event','Conference','2025-03-25','Innovation Hub Dublin',6,6),
('Connor Family Garden Party','Private','2025-03-28','Greenfield Estate Cork',7,7),
('Annual Sales Conference','Corporate','2025-04-01','Tech Hub Dublin',8,8),
('McCarthy Wedding Ceremony','Wedding','2025-04-05','Lakeside Manor Kerry',9,9),
('Digital Marketing Seminar','Conference','2025-04-10','Convention Hall Dublin',10,10);
GO


SELECT * FROM EVENT
```

100 % | 8 | 0 | Ln: 225, Ch: 20 | TABS | CRLF | Windows 1252

Results | Messages

	Event_ID	EventName	EventType	EventDate	Venue	Client_ID	Manager_ID
1	1	O Sullivan Wedding	Wedding	2025-03-01	Grand Hall Dublin	1	1
2	2	Tech Leaders Conference	Conference	2025-03-05	City Convention Centre	2	2
3	3	Byrne 40th Birthday Celebration	Private	2025-03-10	Seaview Hotel Galway	3	3
4	4	Corporate Strategy Meeting	Corporate	2025-03-15	Business Park Limerick	4	4
5	5	Murphy Wedding Reception	Wedding	2025-03-20	Royal Hotel Waterford	5	5
6	6	Startup Networking Event	Conference	2025-03-25	Innovation Hub Dublin	6	6
7	7	Connor Family Garden Party	Private	2025-03-28	Greenfield Estate Cork	7	7
8	8	Annual Sales Conference	Corporate	2025-04-01	Tech Hub Dublin	8	8
9	9	McCarthy Wedding Ceremony	Wedding	2025-04-05	Lakeside Manor Kerry	9	9
10	10	Digital Marketing Seminar	Conference	2025-04-10	Convention Hall Dublin	10	10

Query executed successfully. | localhost\SQLEXPRESS (17.0 ... | KV1725\DELL (83) | ASK_Event_Management | 00:00:00 | Row: 1, Col: 1 | 10 rows

Image 4 – Event Table

● VENDOR:

Stores information on suppliers of event related services.

Primary Columns: vendor_id, vendor_name, service_type, contact_details.

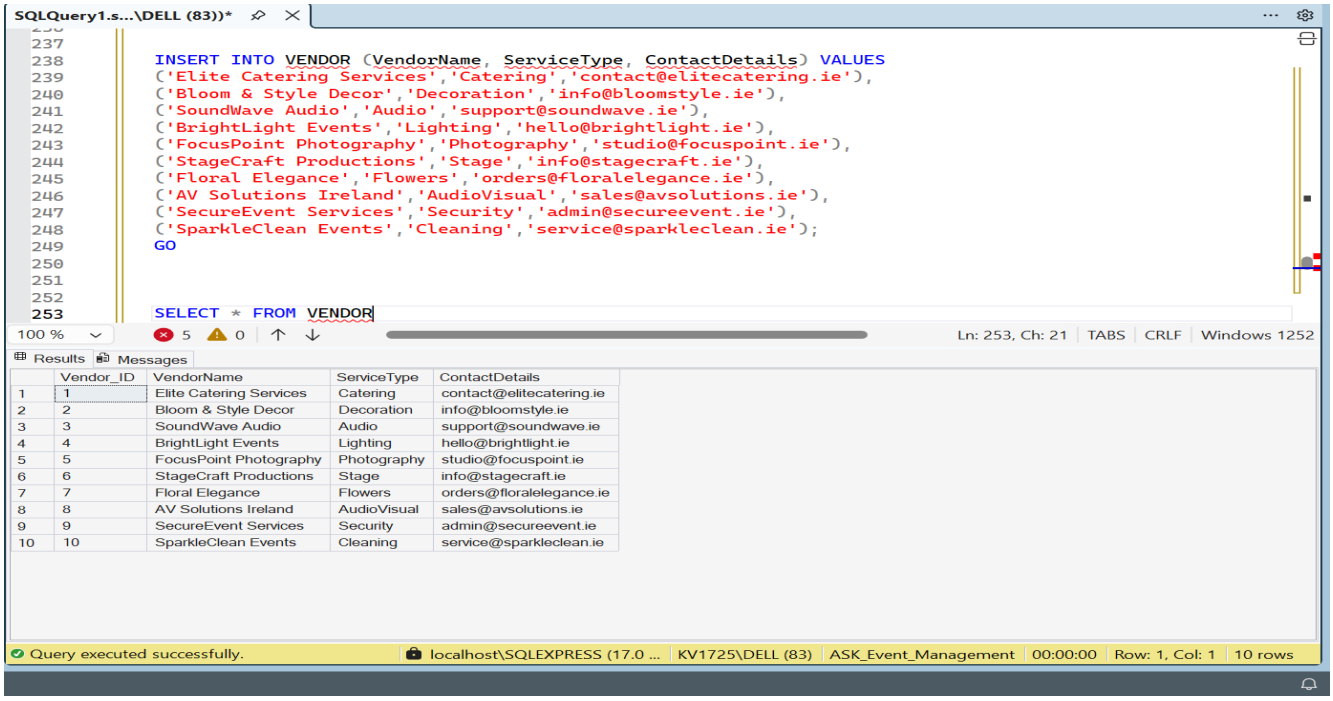


Image 5 – Vendor Table

● EVENT_VENDOR :

Connects events and vendors since most events employ many vendors.

Primary Columns: event_id, vendor_id, service_description.

```sql
INSERT INTO EVENT_VENDOR (Event_ID, Vendor_ID, ServiceDescription) VALUES
(1,1,'Wedding Catering'),
(1,2,'Floral Decoration'),
(2,3,'Conference Audio'),
(2,8,'Projector Setup'),
(3,4,'Lighting Setup'),
(4,5,'Corporate Photography'),
(5,6,'Stage Design'),
(6,7,'Flower Arrangement'),
(7,9,'Event Security'),
(8,10,'Post-event Cleaning');
GO


SELECT * FROM EVENT_VENDOR
```

	Event_ID	Vendor_ID	ServiceDescription
1	1	1	Wedding Catering
2	1	2	Floral Decoration
3	2	3	Conference Audio
4	2	8	Projector Setup
5	3	4	Lighting Setup
6	4	5	Corporate Photography
7	5	6	Stage Design
8	6	7	Flower Arrangement
9	7	9	Event Security
10	8	10	Post-event Cleaning

Image 6 – Event_Vendor Table

● PAYMENT:

Manages accounts of event payments.

Primary Columns: payment_id, event_id, amount, payment_date, status.

```sql
INSERT INTO PAYMENT (Event_ID, Amount, PaymentDate, Status) VALUES
(1,5000,'2025-02-01','Completed'),
(2,3000,'2025-02-05','Pending'),
(3,1500,'2025-02-10','Completed'),
(4,4000,'2025-02-15','Pending'),
(5,6000,'2025-02-20','Completed'),
(6,3500,'2025-02-25','Pending'),
(7,1800,'2025-03-01','Completed'),
(8,4200,'2025-03-05','Pending'),
(9,5200,'2025-03-10','Completed'),
(10,3100,'2025-03-15','Pending');
GO

SELECT * FROM PAYMENT
```

	Payment_ID	Event_ID	Amount	PaymentDate	Status
1	1	1	5000.00	2025-02-01	Completed
2	2	2	3000.00	2025-02-05	Pending
3	3	3	1500.00	2025-02-10	Completed
4	4	4	4000.00	2025-02-15	Pending
5	5	5	6000.00	2025-02-20	Completed
6	6	6	3500.00	2025-02-25	Pending
7	7	7	1800.00	2025-03-01	Completed
8	8	8	4200.00	2025-03-05	Pending
9	9	9	5200.00	2025-03-10	Completed
10	10	10	3100.00	2025-03-15	Pending

Image 7 – Payment Table

● EVENT_DETAILS:

Stores additional event data in XML form.

Primary Columns: details_id, event_id, event_schedule (XML).

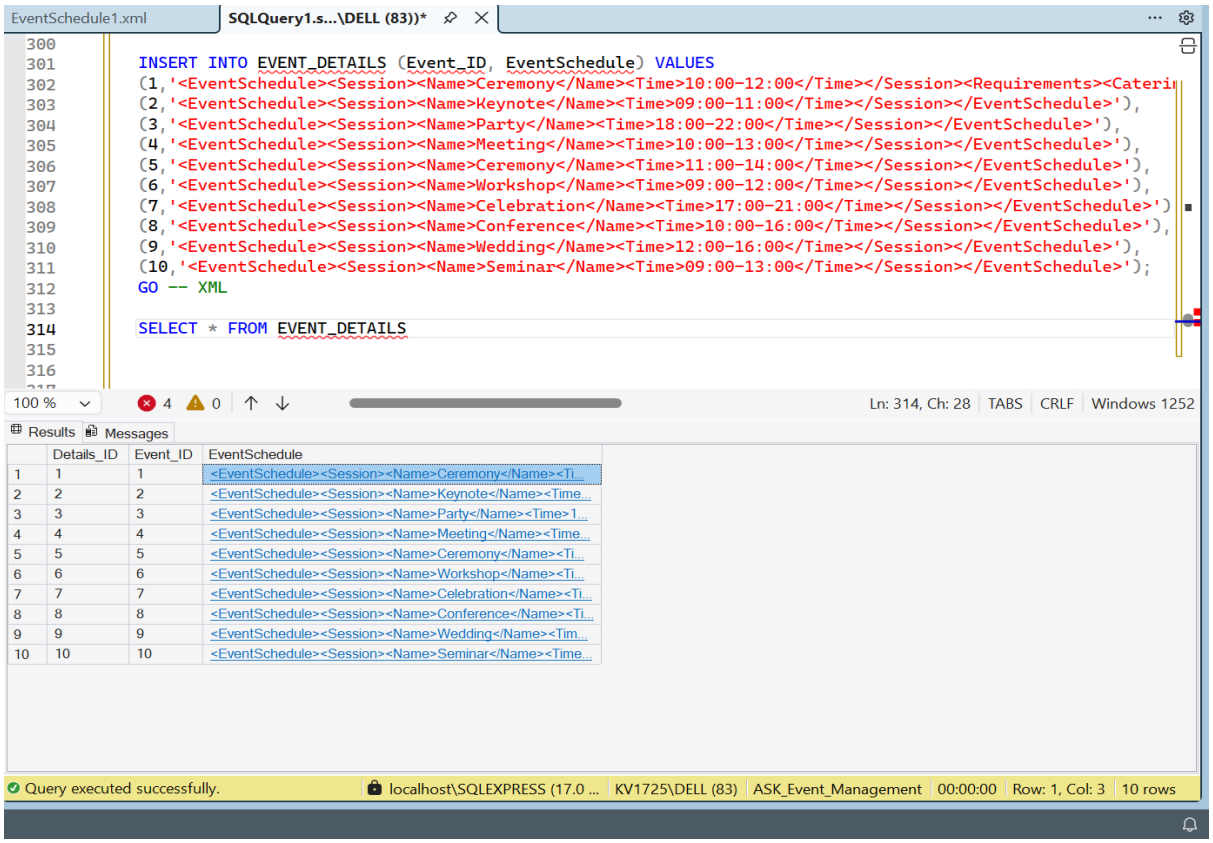


Image 8 – Event_Details Table, inserting as XML data

● PAYMENT_LOG:

Maintains a record of alterations (INSERT, UPDATE, DELETE) that have been made within payment records. Primary Columns: log_id, payment_id, action, log_date, details.

● Key Relationships:

CLIENT ↔ EVENT: A client can book multiple events, but each event belongs to one client.

EMPLOYEE ↔ EVENT: One employee (manager) can manage multiple events.

EVENT ↔ PAYMENT: Each event can have multiple payment records.

EVENT ↔ VENDOR: Events and vendors are linked through the EVENT_VENDOR table (many-to-many).

EVENT ↔ EVENT_DETAILS: Each event can have flexible event details stored using XML.

This schema efficiently organizes client, event, vendor, employee, and payment data for the ASK Event Booking and Management System while maintaining data integrity and consistency.

2.2 Creation of Views

1. EventOverview View:

- This view combines event, client, and payment data to provide a consolidated summary of all events along with their associated clients and payment status.

- The system generates an overview of events known as EventOverview. It retrieves event details of the EVENT table, client details of the CLIENT table and payment details of the PAYMENT table. It employs INNER JOINs and thus only the matching records are displayed. This will allow the managers to get a quick overview of what events will happen and who the clients will be as well as whether the payments have been made or not.

- The ability to have all the data in a single view is convenient during the daily operations, money tracking, and decision making. Event managers do not need to run individual queries on different tables to know the upcoming events, client details and payment status, as this can be done in a short time.

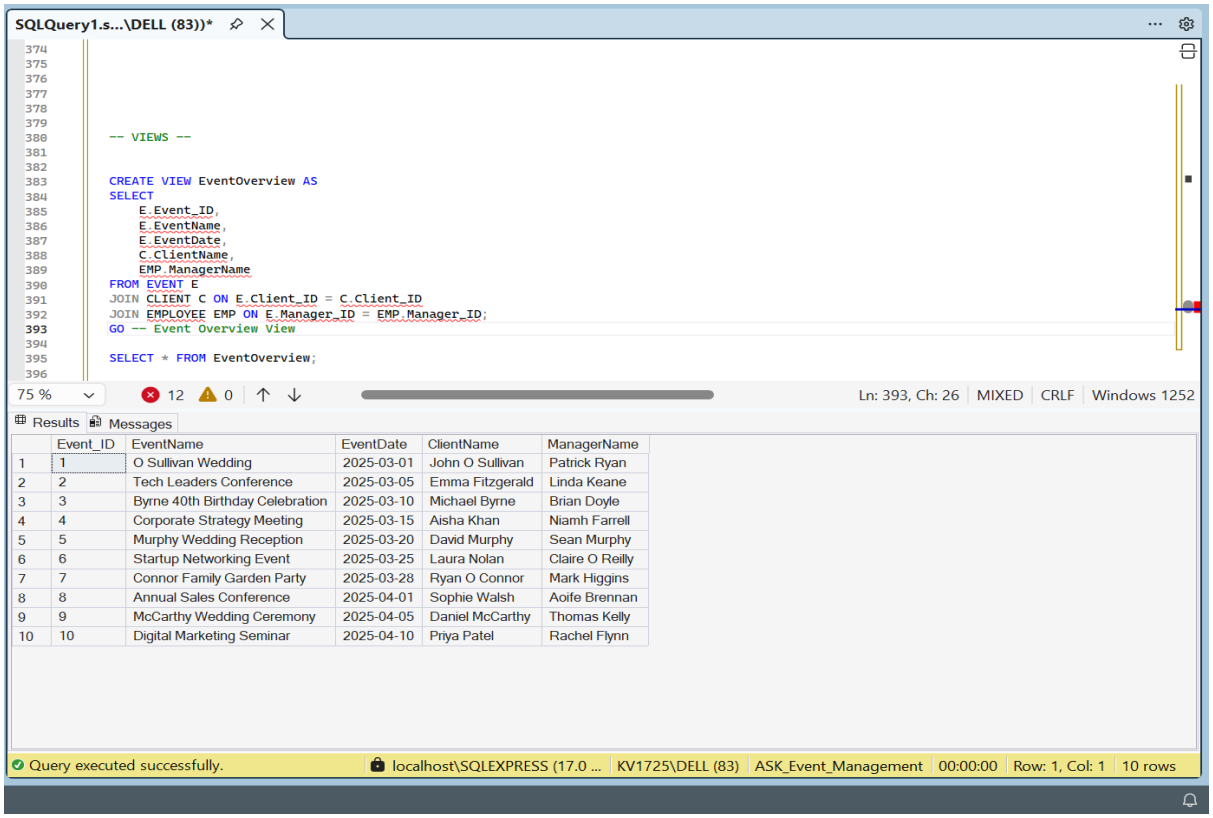


Image 9 – Event Overview VIEW

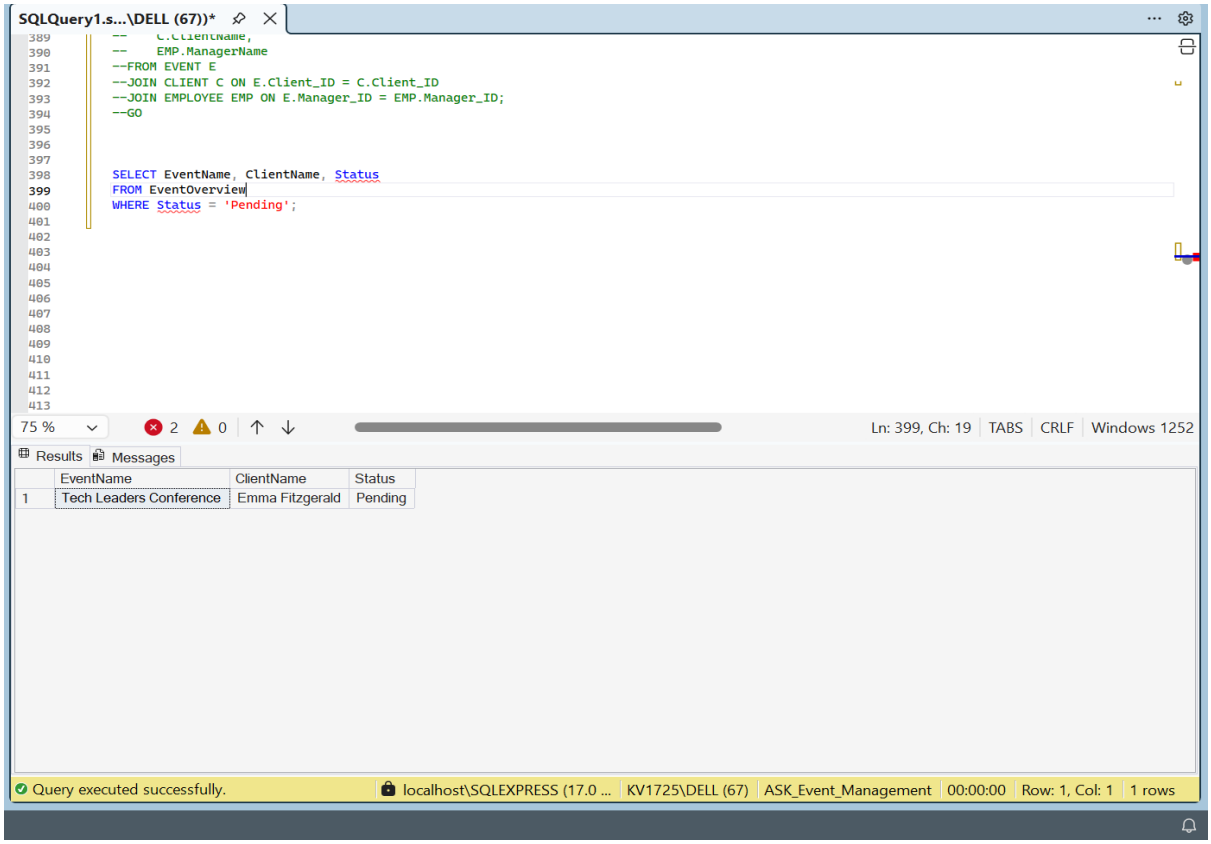


Image 10 – Selecting pending events in Event Overview

SQL Query:

SELECT * FROM EventOverview; (Img 9)

SQL Query:

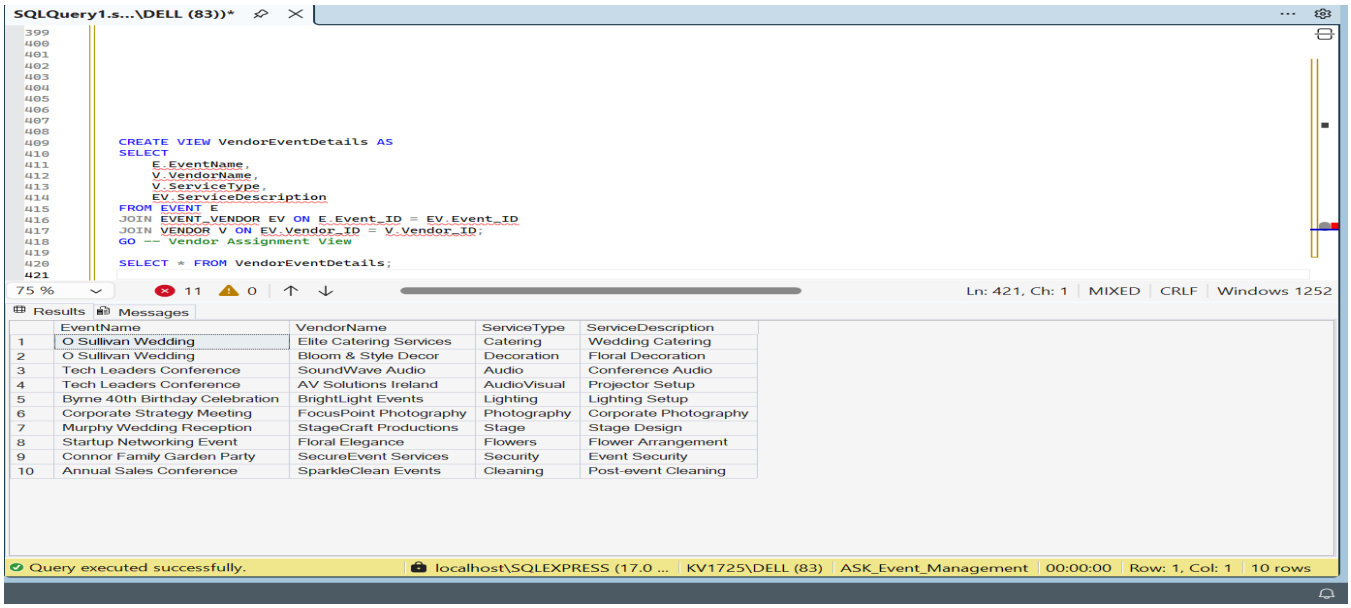
SELECT EventName, ClientName, Status

FROM EventOverview

WHERE Status = 'Pending'; (img 10)

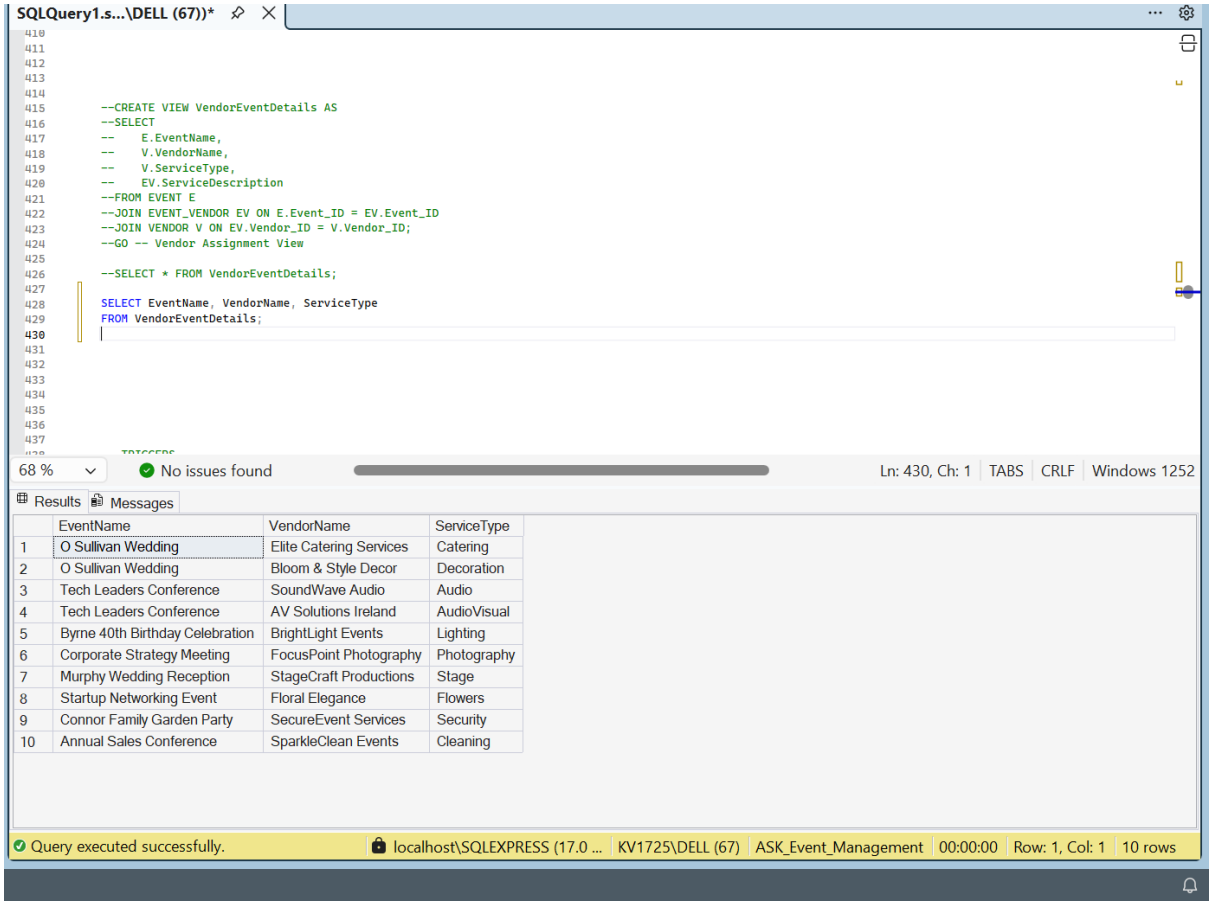
2. VendorAssignment View:

The VendorAssignment view is used to display vendor assignments for each event and to track the services provided by vendors. This view selects event information from the EVENT table and vendor details from the VENDOR table, through the EVENT_VENDOR junction table. It displays the event name, vendor name, service type, and service description. By using this view, the organization can evaluate vendor involvement, manage service allocation, and improve planning and resource distribution for events. 0



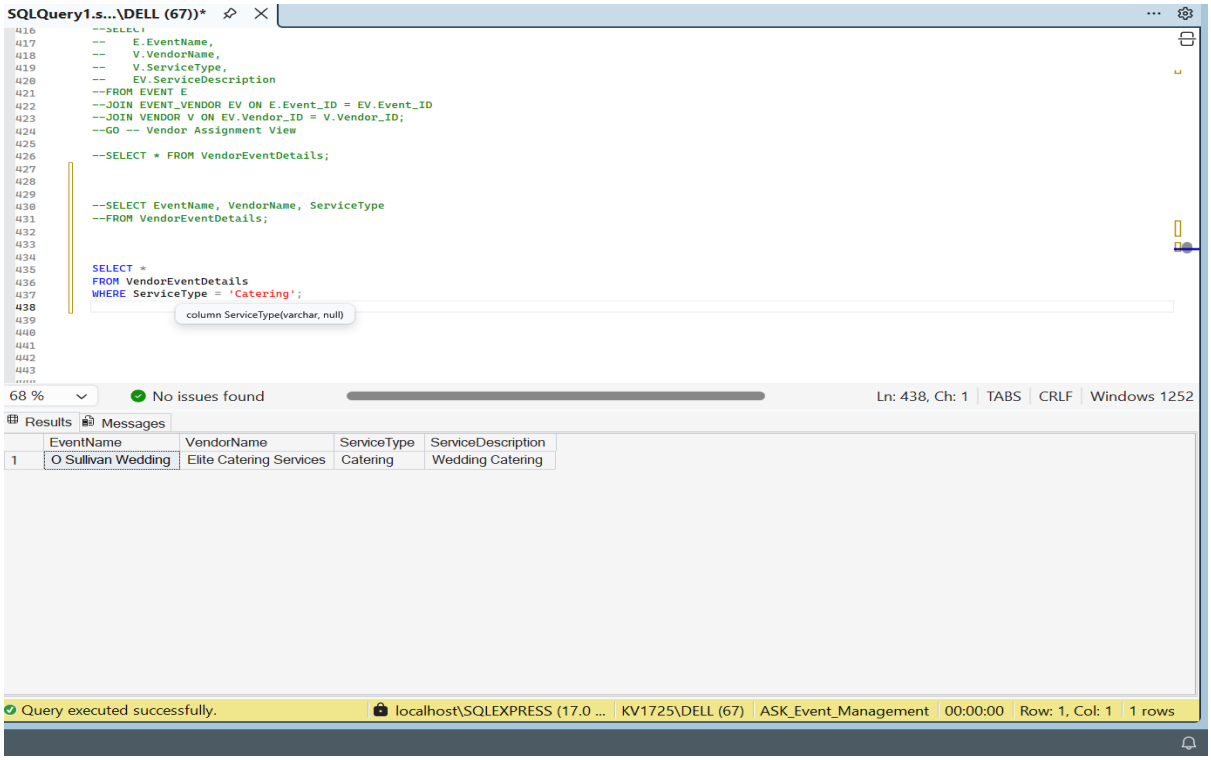
	EventName	VendorName	ServiceType	ServiceDescription
1	O Sullivan Wedding	Elite Catering Services	Catering	Wedding Catering
2	O Sullivan Wedding	Bloom & Style Decor	Decoration	Floral Decoration
3	Tech Leaders Conference	SoundWave Audio	Audio	Conference Audio
4	Tech Leaders Conference	AV Solutions Ireland	AudioVisual	Projector Setup
5	Byrne 40th Birthday Celebration	BrightLight Events	Lighting	Lighting Setup
6	Corporate Strategy Meeting	FocusPoint Photography	Photography	Corporate Photography
7	Murphy Wedding Reception	StageCraft Productions	Stage	Stage Design
8	Startup Networking Event	Floral Elegance	Flowers	Flower Arrangement
9	Connor Family Garden Party	SecureEvent Services	Security	Event Security
10	Annual Sales Conference	SparkleClean Events	Cleaning	Post-event Cleaning

Image 11 – Vendor Assignment VIEW



	EventName	VendorName	ServiceType
1	O Sullivan Wedding	Elite Catering Services	Catering
2	O Sullivan Wedding	Bloom & Style Decor	Decoration
3	Tech Leaders Conference	SoundWave Audio	Audio
4	Tech Leaders Conference	AV Solutions Ireland	AudioVisual
5	Byrne 40th Birthday Celebration	BrightLight Events	Lighting
6	Corporate Strategy Meeting	FocusPoint Photography	Photography
7	Murphy Wedding Reception	StageCraft Productions	Stage
8	Startup Networking Event	Floral Elegance	Flowers
9	Connor Family Garden Party	SecureEvent Services	Security
10	Annual Sales Conference	SparkleClean Events	Cleaning

Image 12 – Getting specific data from Vendor Assignment VIEW



	EventName	VendorName	ServiceType	ServiceDescription
1	O Sullivan Wedding	Elite Catering Services	Catering	Wedding Catering

Image 13 – Getting specific data from Vendor Assignment VIEW

SQL Query:

SELECT * FROM VendorEventDetails; (Img 11)

SQL Query:

SELECT EventName, VendorName, ServiceType FROM VendorEventDetails; (Img 12)

SQL Query:

SELECT * FROM VendorEventDetails WHERE ServiceType = 'Catering'; (Img 13)

2.3 Triggers:

Triggers enhance the ASK Event Booking and Management System functioning as more automatic and implementing the necessary rules and data-uniformity.

a. Prevents the assigning of the same venue by two events simultaneously.

b. Document all the changes (add, change, or delete) in payment records to be verified.

1. trg_UpdateEventStatus

- Automatically updates the event status when a payment is inserted.
- Ensures payment completion is reflected in the EVENT table.
- Reduces manual updates and keeps event data consistent.

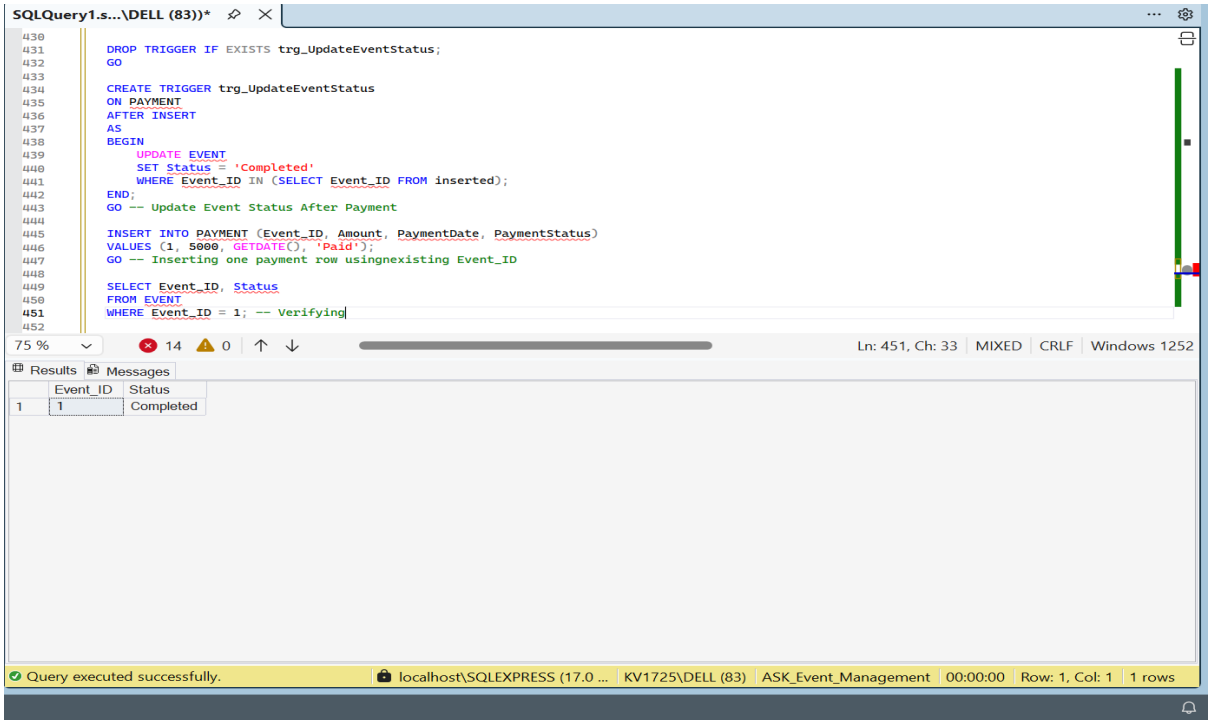


Image 14 – Updating event status using trigger

2. trg_ValidatePayment:

Description:

- Prevents insertion of payments with invalid (negative or zero) amounts.
- Enforces business rules to maintain financial data accuracy.
- Displays an error message when invalid data is entered.

```
485     --    BEGIN
486     --        RAISERROR ('Invalid payment amount. Amount must be greater than zero.', 16, 1);
487     --        RETURN;
488     --    END
489
490     --    INSERT INTO PAYMENT (Event_ID, Amount, PaymentDate, PaymentStatus)
491     --    SELECT Event_ID, Amount, PaymentDate, PaymentStatus
492     --    FROM inserted;
493     --END;
494     --GO -- Trigger to stop invalid payment
495
496     INSERT INTO PAYMENT (Event_ID, Amount, PaymentDate, Status)
497     VALUES (2, 2500, GETDATE(), 'Paid'); -- Valid Payment
498
499     SELECT Event_ID, Status
500     FROM EVENT
501     WHERE Event_ID = 1; -- Verifying
502
503
504
505
506
507
508
509
510     -- FUNCTIONS --
511
512     --CREATE FUNCTION dbo.GetTotalEventPayment (@Event_ID INT)
```

68 % | No issues found | Ln: 503, Ch: 1 | TABS | CRLF | Windows 1252

Results | Messages

	Event_ID	Status
1	1	Completed

Query executed successfully. | localhost\SQLEXPRESS (17.0 ... | KV1725\DELL (67) | ASK_Event_Management | 00:00:00 | Row: 1, Col: 1 | 1 rows

Image 15 – Valid Payment Trigger

SQLQuery1.s...\DELL (83))*

```
450     --FROM EVENT
451     --WHERE Event_ID = 1; -- Verifying
452
453     CREATE TRIGGER trg_ValidatePayment
454     ON PAYMENT
455     INSTEAD OF INSERT
456     AS
457     BEGIN
458         IF EXISTS (SELECT 1 FROM inserted WHERE Amount <= 0)
459         BEGIN
460             RAISERROR ('Invalid payment amount. Amount must be greater than zero.', 16, 1);
461             RETURN;
462         END
463
464         INSERT INTO PAYMENT (Event_ID, Amount, PaymentDate, PaymentStatus)
465         SELECT Event_ID, Amount, PaymentDate, PaymentStatus
466         FROM inserted;
467     END;
468     GO -- Trigger to stop invalid payment
469
470     INSERT INTO PAYMENT (Event_ID, Amount, PaymentDate, PaymentStatus)
471     VALUES (2, -1000, GETDATE(), 'Paid'); -- invalid payment status
472
```

75 % | 16 | 0 | Ln: 468, Ch: 38 | MIXED | CRLF | Windows 1252

Messages

```
Msg 2111, Level 16, State 1, Procedure trg_ValidatePayment, Line 453 [Batch Start Line 0]
Cannot create trigger 'trg_ValidatePayment' on table 'PAYMENT' because an INSTEAD OF INSERT trigger already exists on this object.
Msg 50000, Level 16, State 1, Procedure trg_ValidatePayment, Line 460 [Batch Start Line 468]
Invalid payment amount. Amount must be greater than zero.

(1 row affected)

Completion time: 2025-12-18T23:11:38.0531227+00:00
```

75 % | No issues found | Ln: 9, Ch: 1 | TABS | MIXED | UTF-8 with BOM

Query completed with errors. | localhost\SQLEXPRESS (17.0 ... | KV1725\DELL (83) | ASK_Event_Management | 00:00:00 | Row: 1, Col: 1 | 0 rows

Image 16 – Invalid Payment Trigger

2.4 Stored Procedures and Business Queries:

1. REASON Behind Using Stored Procedures:

- **Data Integrity:** Stored procedures centralize business logic, reducing the risk of invalid or inconsistent data being inserted into the database.
- **Enhanced Security:** Users interact with stored procedures instead of directly accessing tables, preventing unauthorized operations.
- **Improved Performance**: Precompiled execution plans improve query execution speed.
- **Maintainability:** Business rules can be modified easily without changing application-level code.

2. SQL Features Used:

a. JOIN:

JOIN operations are used in multiple parts of the implemented SQL code to retrieve related data from different tables.

Examples include:

- The **EventOverview** view, which uses JOINs between the EVENT, CLIENT, and EMPLOYEE tables to display combined event, client, and manager information. **(Image 9 – Event Overview VIEW uses JOIN)**
- The **VendorEventDetails** view, which uses JOINs between EVENT, EVENT_VENDOR, and VENDOR tables to display vendor assignments for events. **(Image 11 – Vendor Assignment VIEW uses JOIN)**
- Stored procedures such as **GetEventFinancialSummary** also use JOINs between the EVENT and PAYMENT tables to calculate payment-related summaries.

SQLQuery1.s...\DELL (83))*

```sql
-- STORED PROCEDURES --

CREATE PROCEDURE GetEventFinancialSummary
    @MinAmount DECIMAL(10,2)
AS
BEGIN
    SELECT
        E.Event_ID,
        E.EventName,
        COUNT(P.Payment_ID) AS TotalPayments,
        SUM(P.Amount) AS TotalCollected
    FROM EVENT E
    JOIN PAYMENT P ON E.Event_ID = P.Event_ID
    GROUP BY E.Event_ID, E.EventName
    HAVING SUM(P.Amount) >= @MinAmount
    ORDER BY TotalCollected DESC;
END;
GO -- Event financial summary

EXEC GetEventFinancialSummary @MinAmount = 2000;
```

75 % 11 0 Ln: 567, Ch: 1 MIXED CRLF Windows 1252

Results Messages

	Event_ID	EventName	TotalPayments	TotalCollected
1	1	O Sullivan Wedding	3	15000.00
2	2	Tech Leaders Conference	2	6000.00
3	5	Murphy Wedding Reception	1	6000.00
4	9	McCarthy Wedding Ceremony	1	5200.00
5	8	Annual Sales Conference	1	4200.00
6	4	Corporate Strategy Meeting	1	4000.00
7	6	Startup Networking Event	1	3500.00
8	10	Digital Marketing Seminar	1	3100.00

Query executed successfully. localhost\SQLEXPRESS (17.0 ... KV1725\DELL (83) ASK_Event_Management 00:00:00 Row: 1, Col: 1 8 rows

Image 17 – Event Financial Summary

b. GROUP BY and HAVING:

- GROUP BY and HAVING are used in the stored procedure **GetEventFinancialSummary**.
- GROUP BY is used to aggregate payment data by event.
- HAVING is used to filter events based on a minimum total payment amount.
- **Image 17 – Event Financial Summary uses GROUP BY and HAVING**

c. User-defined functions:

User-defined functions are implemented to improve code reusability and modularity.

Examples include:

- **GetTotalEventPayment**, which calculates the total payment amount for a given event.
- **GetVendorCountByEvent**, which returns the number of vendors assigned to a specific event. 70

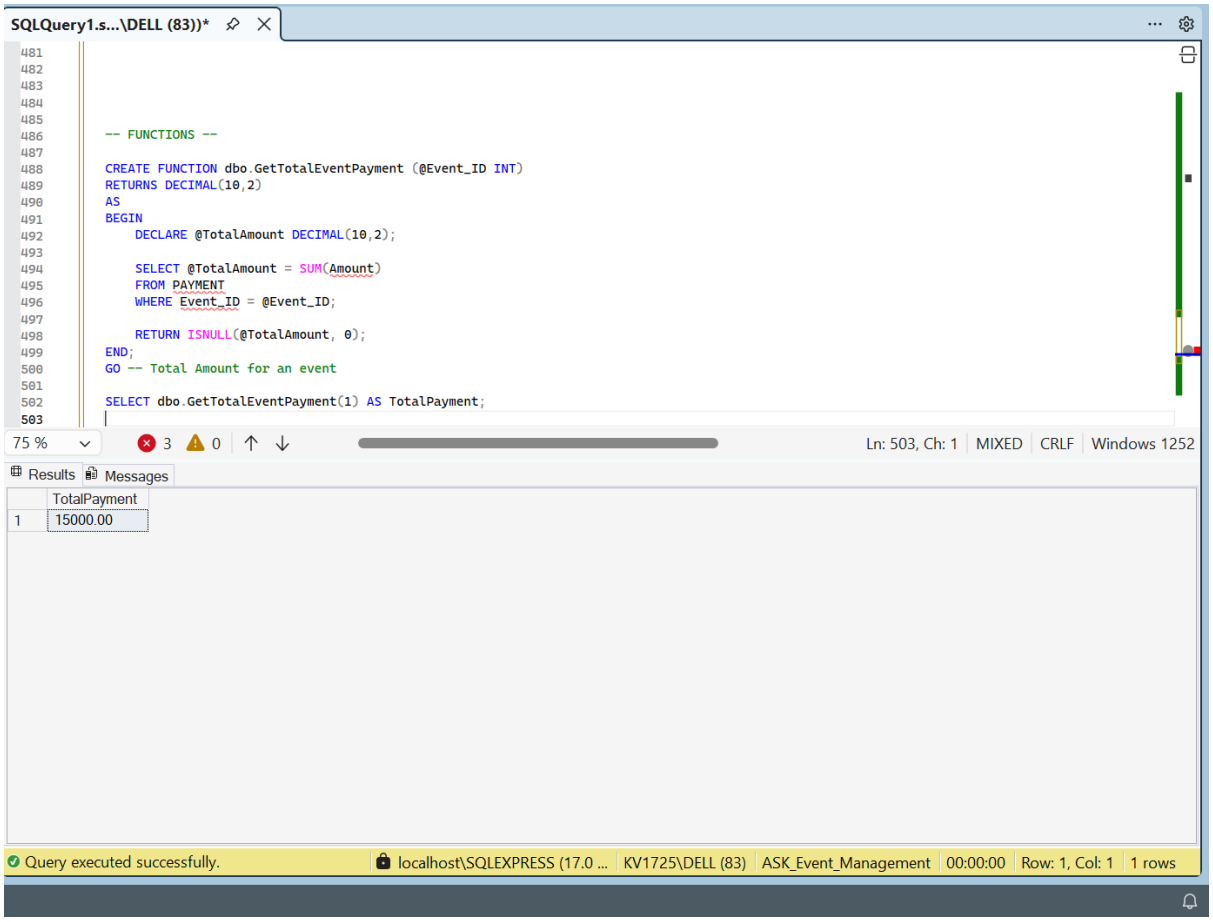


Image 18 – Total Event Payment

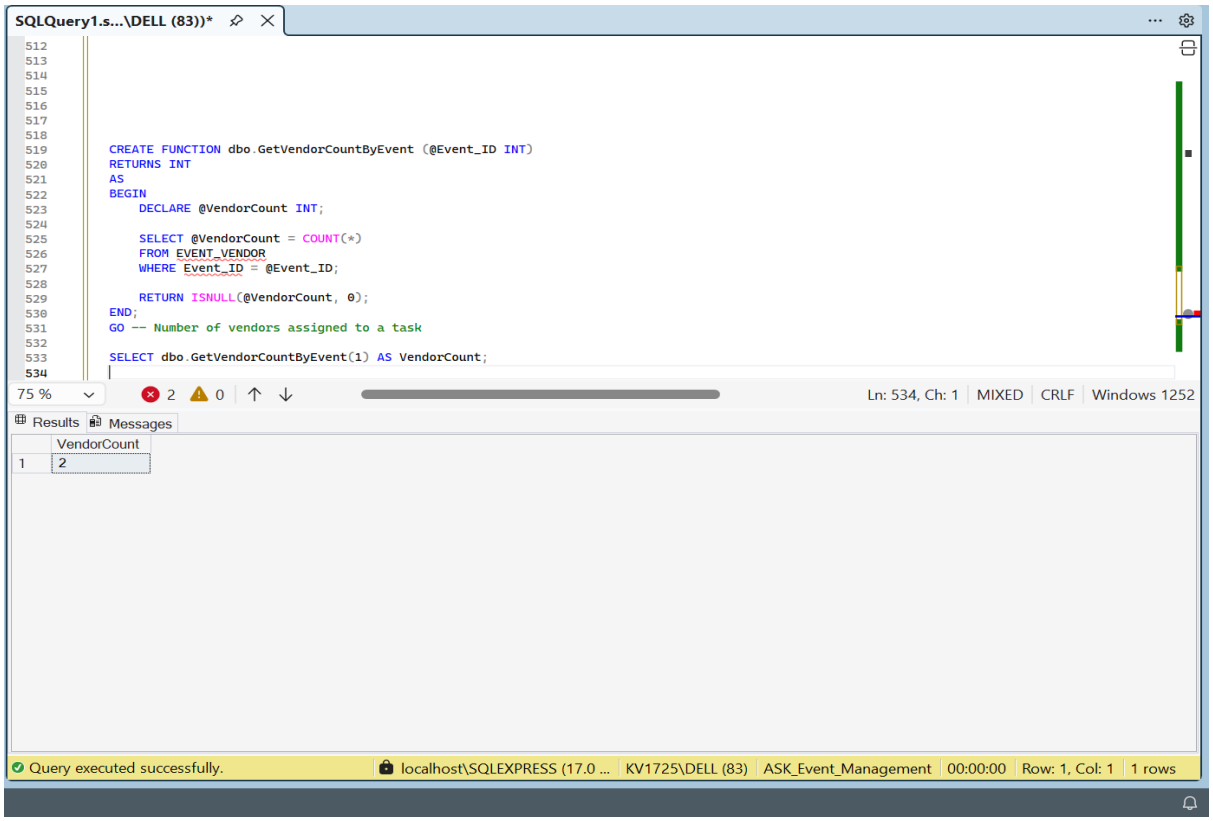


Image 19 – Vendor Count by Event

3. HYBRID DATABASE FEATURES AND XML IMPLEMENTATION:

3.1 Justification for Using XML Data Type:

- XML is an appropriate choice of information storage that cannot be totally determined in the ASK Event Booking and Management System. XML is capable of dealing with still hierarchically organized data that is able to change with every event. Among the information that never change such as client and event IDs, and those which can be changed e.g. schedules, sessions and special requests.

- Under XML, the database is able to store the complex and dynamic details of events without having to store the database tables. 0

1. Tables Where XML Is Applied:

a. EVENT Table:

Use Case:

- Each event will need different requirements.
- Like some events need different food, some events need different stage arrangements, some need different seating arrangements.
- So, these changes differ in a huge amount from event to event

Reason for XML:

- XML helps to organize these changes in a format where the most required needs are stored first and other needs.
- It ranks the needs according to their demand.

Refer to Image 8 – Event_Details Table, inserting as XML data

XML Code:

```
<EventSchedule>
  <Session>
    <Name>Ceremony</Name>
    <Time>10:00-12:00</Time>
  </Session>
```

</EventSchedule>

- This XML structure allows flexible storage of events.
- Event employees can also access the even-specific requirements with this structure.
- Overall, XML in ASK Events helps to stored data in a flexible way and also allow easy access to event data.
- It also stores data in an understandable manner. 0

3.2 XML Data Operations:

Developing XML:

- In this we are Generating XML data from EVENT_DETAILS table.
- This system generates XML data to display event schedules and other event details stored in the database.
- XML allow details like session names, timings and special need by the customer to be stored in a hierarchical format.
- This makes the data easier to store, transfer and refer.

Refer to Image 8 – Event_Details Table, inserting as XML data

3.3 Retrieving Data from XML Fields:

- If we want to extract specific information like event_schedule or session name from the EVENT_DETAILS table, we can use XQuery operations to easily do that.

XML Code:

```
SELECT
EventID,
EventSchedule.value('(/EventSchedule/Session/Name)[1]','VARCHAR(100)')AS SessionName,
EventSchedule.value('(/EventSchedule/Session/Time)[1]', 'VARCHAR(50)') AS SessionTime
FROM EVENT_DETAILS
```

- This query gets the session name and time for EVENT_DETAILS Table.

- In this event managers can retrieve the data directly from database without additional tables.

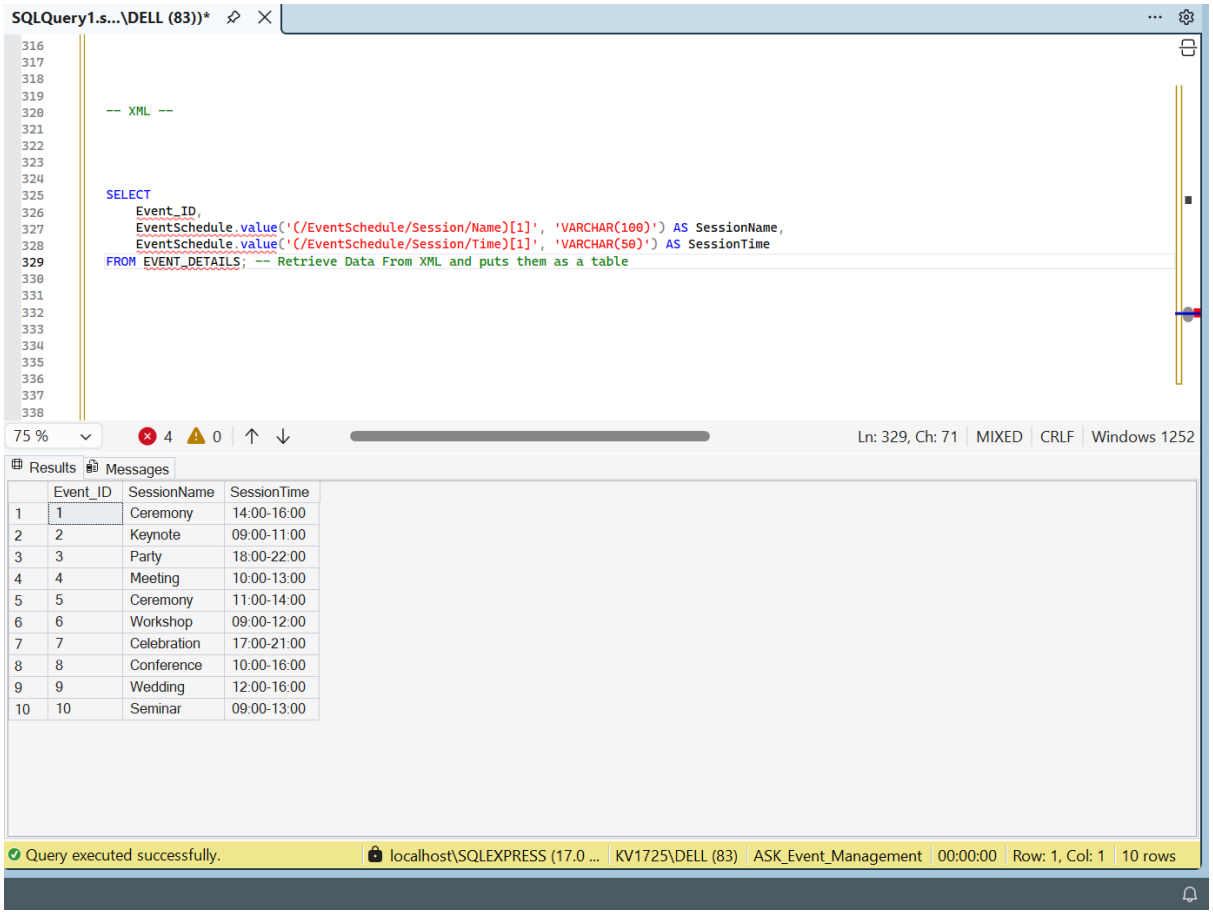


Image 20 – Retrieving Data from XML

3.4 Modifying XML Data:

- If there is any changes in the schedule or requirements, then the XML data stored in EVENT_DETAILS table can be updated without modifying the database.

XML Code:

```
UPDATE EVENT_DETAILS
SET EventSchedule.modify(
'replace value of (/EventSchedule/Session/Time/text())[1]
 with "14:00 - 16:00"'
)
WHERE EventID = 1;
```

- This code allows event managers to update changes in timings or events in the XML document without modifying the whole database.

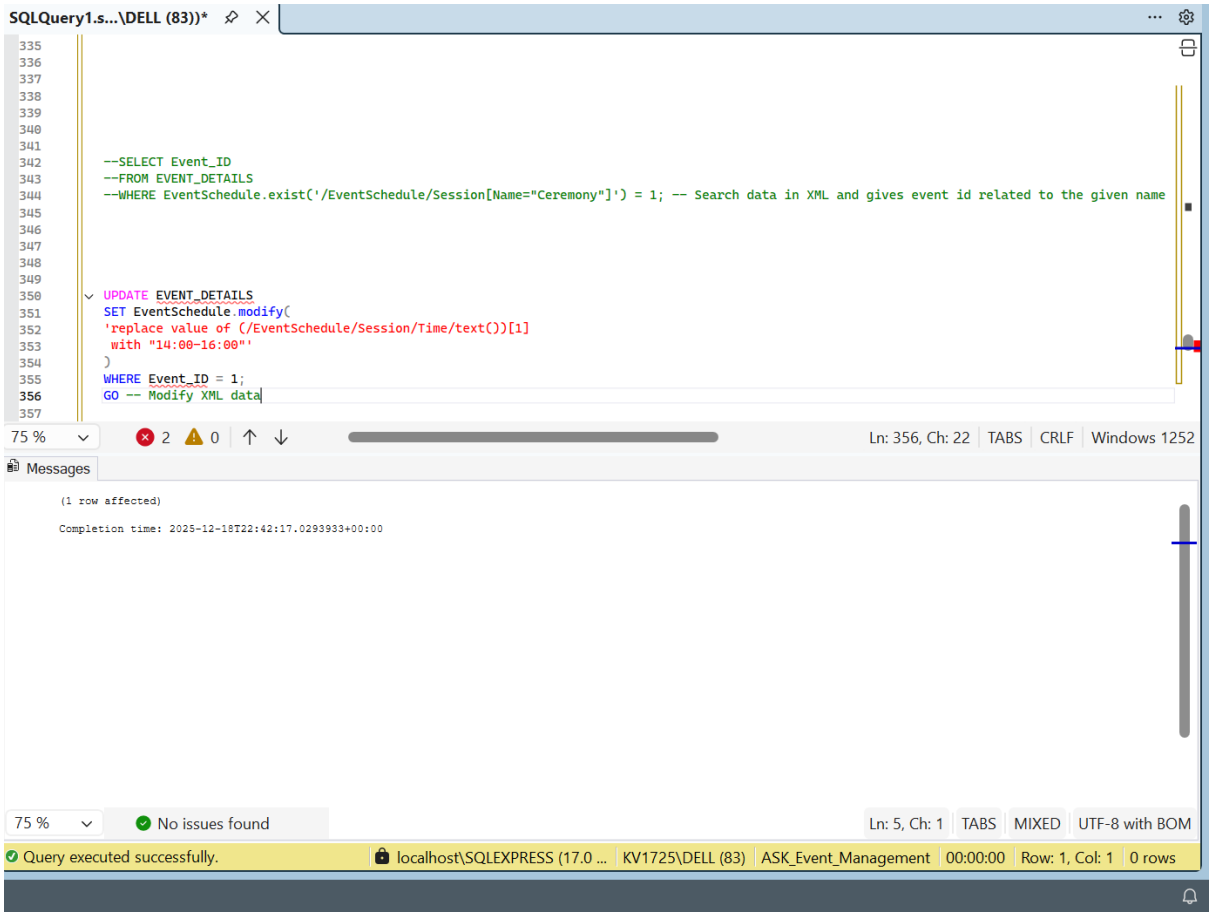


Image 21 – Modifying XML Data

3.5 Searching within XML Data:

- We can use exist() function to search for specific information in EVENT_DETAILS Table.

XML Code:

```
SELECT Event_ID
FROM EVENT_DETAILS
WHERE EventSchedule.exist('/EventSchedule/Session[Name="Ceremony"]') = 1;
```

Using this query, event managers can search for events based on specific session names stored inside the XML data, such as finding all events that include a **Ceremony session.**

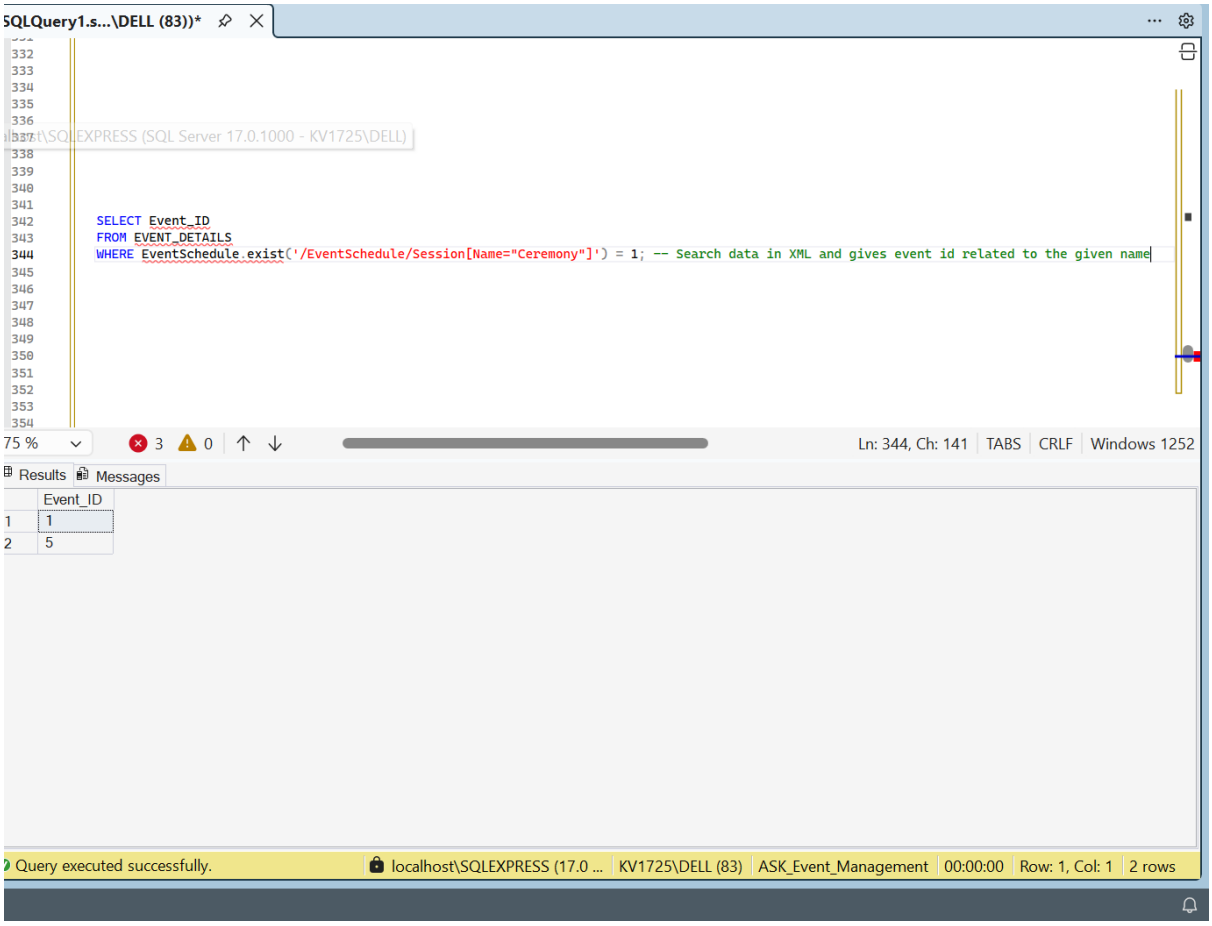


Image 22 – Searching XML Data

4. CHALLENGES AND SOLUTIONS:

There were several challenges that we faced during the development of ASK Events

1. Relationship Management among Tables.

Challenge:

- It is a system with numerous connected tables including CLIENT, EVENT, Payment, EMPLOYEE and Vendor. It was difficult to maintain the connection between these tables.

Solution:

- The links between the tables were maintained with the help of the foreign key constraints. As an example, the EVENT table (Client_ID) refers to the CLIENT table. The

many to many relationships between vendors and events was handled in the EVENT_VENDOR table.

2. Venue Double booking.

Challenge:

- It was a risk of having two or more events in the same location on the same day.

Solution:

- An insertion or update of an event was blocked in case the venue was already booked on that date. This automatically averted venue issues.

3. Managing Payments and Payments Acceptance.

Challenge:

- Financial records would also be disturbed by wrong payments entries such as negative values.

Solution:

- Checks on the amount of payment were in the form of triggers prior to adding them. The event status was applied as Completed automatically when a valid payment had been made.

4. XML Data of Event Schedules.

Challenge:

- The schedules of events are varied to each event and do not easily fit into predetermined columns.

Solution:

- The EVENT_DETAILS table was created with XML data type to contain the flexible event schedule. This schedule data could be easily inserted, searched, updated and

retrieved using XML queries.

5. INNOVATION:

- ASK Event Booking and Management System possess numerous features to simplify the database and improve its speed.
- We created views and stored procedures to allow users to get event information, vendor assignments and payment information quickly without having to write hard SQL. This makes it easy to use this system by managers and admins.
- Jobs of importance were added to be automated through the addition of triggers. As an illustration, they eliminate the problem of double booking and ensure payment changes are executed appropriately.
- Audit logging follows changes on important data such as payment.
- XML is used to store such flexible event data as schedules and special needs. This allows the system to store numerous varieties of events without frequent changes to the database scheme.

6. CONCLUSION:

6.1 Overview of the Implemented System:

- ASK Event booking and management system is a database-based system that is used to record clients, events, venues, vendors, staff, and payments. It takes the form of tables that are interrelated and incorporates rules, triggers and scripts to maintain the data right.
- Triggers and scripts are automatic to minimize manualized processes and prevent issues such as the two-booking of a venue or poor payment record. XML does not restrict the data to a rigid format, but rather it contains event-specific data like schedules and special requests which means that the database can support numerous types of events without having to make regular adjustments to the structure.
- The combination of database solutions enables the system to expand, provide reports and audits through payment logs and views as well as enable users to view current data in real time.

- **Operational Effectiveness:** Trigger and stored procedures allow automation of operations, decreasing human work and accelerating processing.

- **Scalability:** The normalised schema is favourable to business development and it can manage additional event data.

- **Reporting and Auditing:** There is a possibility of accurate reporting and clear financial tracking using the Payment logs and summary views.

- **Real-time access:** Users are able to see up to date event and client and payment data at any time.

- **Data Accuracy:** Constraints and validation rules maintain all the tables consistent and reliable.

- All in all, the ASK Events database, with these characteristics, is capable of becoming expandable, adaptable and future-proof.

7. REFERRENCES:

7.1 Book References:

- Connolly, T.M. and Begg, C.E., 2005. Database systems: a practical approach to design, implementation, and management. Pearson Education.

7.2 Website References:

- https://www.geeksforgeeks.org/dbms/how-to-design-a-database-for-event-management/ - Geeks for Geeks
- https://webxlearner.com/event-management-system-project/ - WEBX Learner
- Event Management Database Part 1 by Filasco Design Studio – Youtube.com

8. Table Creation images:

```
--GO -- Use DB



-- TABLE CREATION --


CREATE TABLE CLIENT (
    Client_ID INT IDENTITY PRIMARY KEY,
    ClientName VARCHAR(100) NOT NULL,
    Email VARCHAR(100) UNIQUE,
    Phone VARCHAR(20),
    Address VARCHAR(200)
);
GO  -- Client
```

```
Commands completed successfully.

Completion time: 2025-12-18T22:19:29.6097241+00:00
```

Image 23 – Client Table Creation

```
--);
--GO  -- Client



CREATE TABLE EMPLOYEE (
    Manager_ID INT IDENTITY PRIMARY KEY,
    ManagerName VARCHAR(100) NOT NULL,
    Email VARCHAR(100),
    Phone VARCHAR(20),
    Position VARCHAR(50)
);
GO -- Employee
```

```
Commands completed successfully.

Completion time: 2025-12-18T22:20:11.8788490+00:00
```

Image 24 – Employee Table Creation

```sql
CREATE TABLE EVENT (
    Event_ID INT IDENTITY PRIMARY KEY,
    EventName VARCHAR(100),
    EventType VARCHAR(50),
    EventDate DATE,
    Venue VARCHAR(100),
    Client_ID INT,
    Manager_ID INT,
    CONSTRAINT FK_Event_Client
        FOREIGN KEY (Client_ID)
        REFERENCES CLIENT(Client_ID),
    CONSTRAINT FK_Event_Manager
        FOREIGN KEY (Manager_ID)
        REFERENCES EMPLOYEE(Manager_ID)
);
GO  -- Event
```

```
Commands completed successfully.

Completion time: 2025-12-18T22:20:52.9249589+00:00
```

Image 25 – Event Table Creation

```sql
--);
--GO  -- Event

CREATE TABLE VENDOR (
    Vendor_ID INT IDENTITY PRIMARY KEY,
    VendorName VARCHAR(100),
    ServiceType VARCHAR(50),
    ContactDetails VARCHAR(100)
);
GO  -- Vendor
```

```
Commands completed successfully.

Completion time: 2025-12-18T22:20:52.9249589+00:00
```

Image 26 – Vendor Table Creation

```sql
CREATE TABLE EVENT_VENDOR (
    Event_ID INT,
    Vendor_ID INT,
    ServiceDescription VARCHAR(100),
    CONSTRAINT PK_EventVendor PRIMARY KEY (Event_ID, Vendor_ID),
    CONSTRAINT FK_EV_Event
        FOREIGN KEY (Event_ID) REFERENCES EVENT(Event_ID),
    CONSTRAINT FK_EV_Vendor
        FOREIGN KEY (Vendor_ID) REFERENCES VENDOR(Vendor_ID)
);
GO -- Junction Table
```

```
Commands completed successfully.

Completion time: 2025-12-18T22:22:49.6062014+00:00
```

Image 27 – EVENT_Vendor Table Creation

```sql
CREATE TABLE PAYMENT (
    Payment_ID INT IDENTITY PRIMARY KEY,
    Event_ID INT NOT NULL,
    Amount DECIMAL(10,2),
    PaymentDate DATE,
    Status VARCHAR(20),
    CONSTRAINT FK_Payment_Event
        FOREIGN KEY (Event_ID) REFERENCES EVENT(Event_ID)
);
GO -- Payment
```

```
Commands completed successfully.

Completion time: 2025-12-18T22:23:30.7937267+00:00
```

Image 28 – PAYMENT Table Creation

```sql
--);
--GO -- Payment

CREATE TABLE EVENT_DETAILS (
    Details_ID INT IDENTITY PRIMARY KEY,
    Event_ID INT NOT NULL,
    EventSchedule XML,
    CONSTRAINT FK_EventDetails_Event
        FOREIGN KEY (Event_ID) REFERENCES EVENT(Event_ID)
);
GO -- XML Table
```

```
Commands completed successfully.

Completion time: 2025-12-18T22:24:02.9255958+00:00
```

Image 29 – EVENT_DETAILS Table Creation

```sql
CREATE TABLE PAYMENT_LOG (
    Log_ID INT IDENTITY PRIMARY KEY,
    Payment_ID INT,
    ActionType VARCHAR(20),
    LogDate DATETIME DEFAULT GETDATE()
);
GO -- Audit Trigger
```

```
Commands completed successfully.

Completion time: 2025-12-18T22:24:51.8365386+00:00
```

Image 30 – PAYMENT_LOG Table Creation

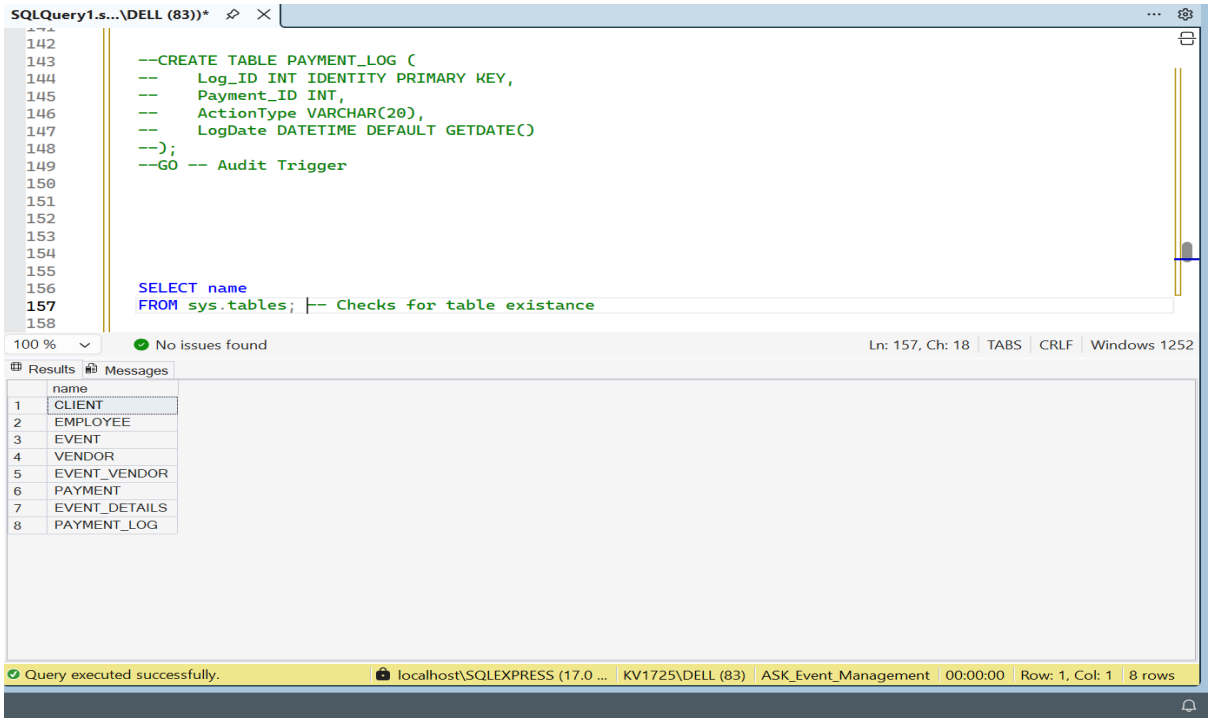


Image 31 – Verifying Table Existence

9. ER Diagram:

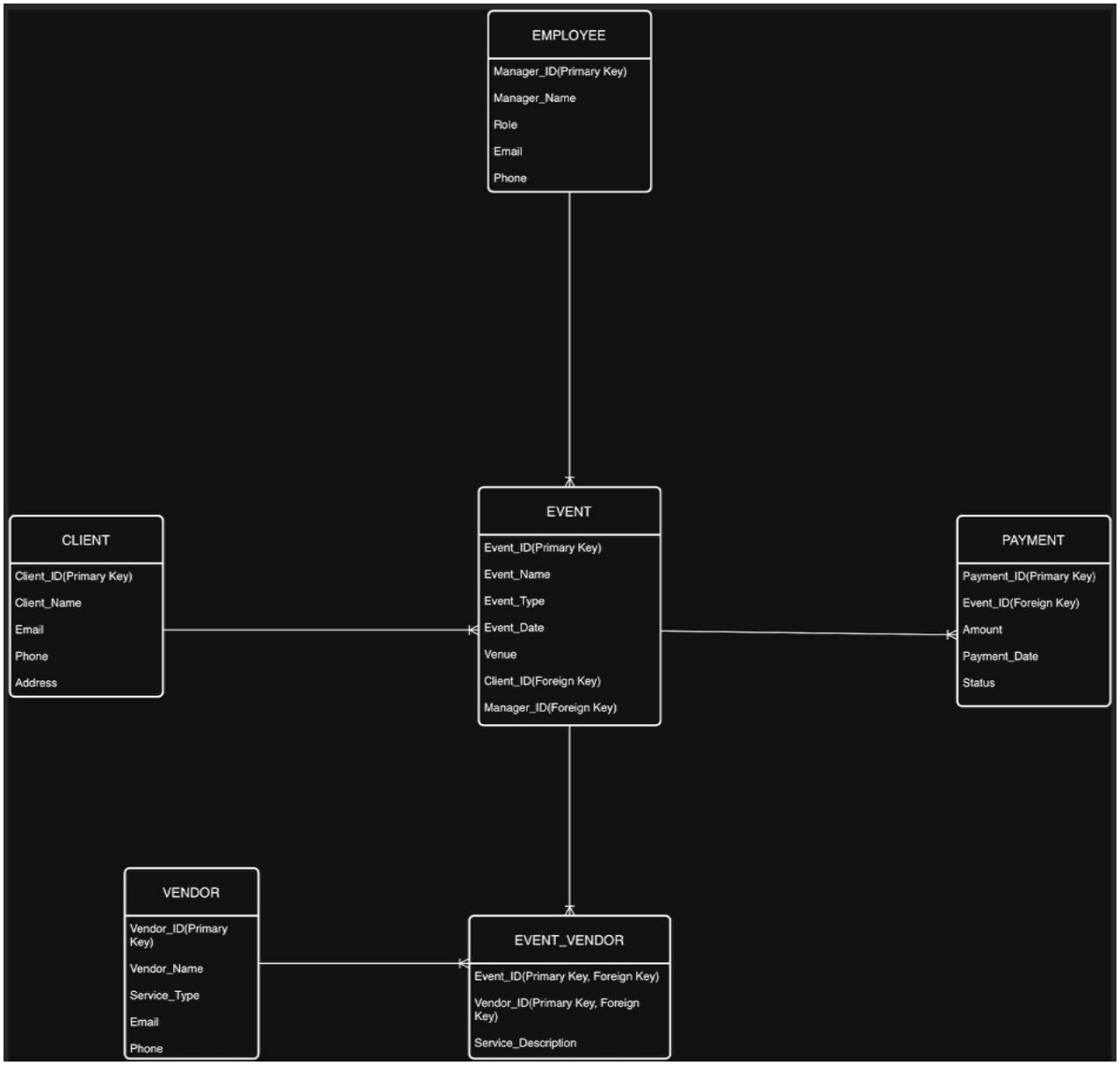


Image 32 – ASK (ER Diagram)